

Contents

1	Problem Statement	3
2	Introduction & Objectives	3
2.1	Introduction	3
2.2	Project Objectives	4
3	Assumptions & Match Rules	4
3.1	General Match Constraints	4
3.2	Operating System Equivalencies	5
3.3	Dynamic Resource Limits & Match Phases	6
4	System Architecture	6
4.1	High-Level Overview	6
4.2	System Modules and File Structure	7
4.3	Concurrency Primitives and Libraries	7
5	Thread Design & Lifecycle	8
5.1	The Umpire Thread (Scheduler & State Manager)	8
5.2	The Bowler Thread (The Producer)	10
5.3	The Batsman Threads (The Consumer)	12
5.4	The Fielder Threads (Interrupt Handlers)	14
6	Thread Synchronization	15
6.1	The State Machine Model	15
6.2	Condition Variables and Signalling	16
6.3	The Game Loop Walkthrough	16
7	Critical Sections & Shared Memory	17
7.1	Identification of Shared Memory	17
7.2	Mutex Lock Implementation	17
8	Resource Scheduling	18
8.1	Round-Robin & Resource Quotas (Overs 1–19)	18
8.2	Priority Preemption (The Death Over)	19

9	Deadlock Detection & Run-Out Handling	19
9.1	Trigger Condition	20
9.2	The Circular Wait Scenario	20
9.3	The Resource Allocation Graph (RAG)	21
9.4	Cycle Detection (Snapshot Technique)	21
9.5	Deadlock Resolution (Process Termination)	22
10	Execution Logging & Gantt Chart	22
10.1	Thread-Safe Logging	22
10.2	High-Resolution Timestamping	23
10.3	Gantt Chart Output	23
11	Performance Analysis (FCFS vs. SJF Scheduling)	25
11.1	First-Come, First-Served (FCFS)	25
11.2	Shortest Job First (SJF)	25
11.3	Wait Time Comparison	26
11.4	Conclusion & Wait Time Reduction	26
12	Conclusion	26

1. Problem Statement

Modern operating systems rely heavily on process scheduling, inter-process communication, thread synchronization, and deadlock management to ensure that multiple concurrent processes run smoothly without crashing or corrupting data. The objective of this project is to model and simulate a real-world, highly concurrent environment—a T20 Cricket World Cup match—using core Operating System principles.

In this simulation, the entities of a cricket match are mapped to system resources and concurrent processes:

- **Players (Bowlers, Batsmen, Fielders)** act as independent, concurrent Threads.
- **The Pitch** represents a shared Critical Section that must be carefully protected to avoid race conditions and deadlocks.
- **The Umpire** acts as the CPU/Resource Scheduler, orchestrating the flow of the match, enforcing constraints, and ensuring safe context switching between threads.

The system must handle complex synchronizations where multiple fielders and batsmen interact simultaneously. Furthermore, the simulation must intelligently handle resource allocation conflicts, specifically modeling a “Run-Out” as a Circular Wait Deadlock (where two batsmen threads compete for the same crease resource). The ultimate goal is to produce a fully synchronized, multi-threaded simulator that logs ball-by-ball execution, provides a deterministic Gantt chart of thread execution, and allows for the performance analysis of different scheduling algorithms (such as FCFS vs. SJF) on player wait times.

2. Introduction & Objectives

2.1 Introduction

The simulation of real-world concurrent events is a classic and powerful application of Operating System (OS) principles. A live cricket match is inherently highly concurrent: multiple entities—bowlers, batsmen, fielders, and umpires—operate simultaneously, react to shared events, and compete for shared physical space.

This project presents a highly synchronized, multi-threaded C++ simulator. By treating

the cricket pitch as a shared system resource, the players as concurrent POSIX threads (pthreads), and the rules of the game as system synchronization primitives, this simulator bridges the gap between abstract computer science theories and real-world physical dynamics. The system handles complex state transitions and orchestrates 14 active threads per ball without data corruption or fatal system deadlocks.

2.2 Project Objectives

The primary goal of this project is to architect a concurrent system that fulfils the following specific OS objectives:

1. **Entity-to-Thread Mapping:** Implement independent worker threads for the Bowler, Batsmen, Fielders, and the Umpire.
2. **Strict Thread Synchronization:** Protect the “Pitch” and global match variables using Mutex Locks (`pthread_mutex_t`) and Condition Variables (`pthread_cond_t`), ensuring 100% data integrity and preventing race conditions.
3. **Resource Scheduling:** Design the Umpire thread as a central CPU scheduler that enforces resource quotas (a maximum 4-over limit per bowler) using Round-Robin and Priority Preemption scheduling. Evaluate scheduling strategies (FCFS vs. SJF) on the wait times of middle-order batsmen threads.
4. **Deadlock Detection & Resolution:** Model a cricket “Run-Out” as a classic OS Circular Wait Deadlock. Dynamically detect this using a Resource Allocation Graph (RAG) and Depth-First Search (DFS) cycle detection, and resolve it by terminating the victim thread.
5. **Execution Logging & Visualization:** Safely log concurrent thread execution using microsecond timestamps and generate a deterministic Gantt Chart visualizing which thread held the CPU/Pitch resource at any given moment.

3. Assumptions & Match Rules

3.1 General Match Constraints

- **Format & Teams:** Standard T20 format (maximum 20 overs or 120 legal deliveries) between two fixed 11-player teams (India vs. Australia).

- **Toss:** A randomized “Third Umpire” algorithm determines the toss winner and batting choice before thread creation.
- **Termination Conditions:** An innings terminates when either 20 overs are completed, 10 wickets fall, or the batting team successfully chases the target score.
- **Maximum Bowlers:** Only 6 bowlers are used per innings; no No-balls are modelled; the death over is defined as the 20th over only.
- **Probabilities:** All outcome probabilities are hand-tuned based on real T20 averages—not player-specific stats. Pressure increases aggression via `compute_pressure()`.
- **Run-Out Possibility:** There is a possibility of run-out on every run taken.

3.2 Operating System Equivalencies

- **Threads:** Every player on the field (1 Bowler, 2 Batsmen, 10 Fielders) and the Umpire are modeled as independent POSIX threads (`pthreads`).
- **Critical Section:** The “Pitch” and the shared scoreboard state act as the critical section, protected by mutexes. Only one thread can modify it at a time.
- **CPU Scheduler:** The Umpire thread acts as the central scheduler, using state machines to block/unblock player threads.
- **Deadlock Model:** A “Run-Out” is explicitly modeled as a Circular Wait Deadlock (e.g., Batsman A holds Crease 1 requesting Crease 2, while Batsman B holds Crease 2 requesting Crease 1).

Table 1: Cricket Entity $\longleftrightarrow$ OS Concept Mapping

Cricket Entity	OS Concept
Players (Batsman, Bowler, Fielder)	Threads (pthread)
Pitch	Critical Section
Crease (max 2 batsmen)	Semaphore (capacity = 2) + Mutex per end
Umpire (game loop)	Resource Scheduler / Kernel
Run-Out Scenario	Deadlock Detection & Resolution
Global Score	Shared Resource (Mutex-protected)
Over Transition	Context Switch (Round Robin)
Fielder Activation	Condition Variable (BALL IN PLAY)
Batting Order	Scheduling Policy (FCFS / SJF)
Death Overs	Priority Scheduling

3.3 Dynamic Resource Limits & Match Phases

- **Resource Quotas:** The Umpire enforces a strict 4-over maximum per bowler. Once reached, the scheduler bypasses them for the rest of the innings.
- **The Death Over (20th Over):** Normal round-robin rotation is suspended and the Umpire preemptively assigns the final over to the available bowler with the highest specialist priority rating.

4. System Architecture

4.1 High-Level Overview

The cricket simulator is designed as a highly concurrent, state-driven C++ application. Unlike a traditional sequential program, this system relies entirely on Thread Synchronization to drive the game forward.

At the core of the architecture is the *Shared Memory Model*. The “Pitch” and the “Scoreboard” are represented by a set of global variables. Instead of passing messages between players, all 14 active threads continuously run in the background, observing and

modifying this shared global state under the strict protection of Mutex locks. The Umpire thread acts as the central orchestrator (CPU Scheduler), transitioning a global `state` variable that dictates which thread group is allowed to execute at any given microsecond.

4.2 System Modules and File Structure

- **State & Configuration**

- `headers.hpp` — All structural definitions, enumerations (`Outcome`, `NodeType`), and external declarations.
- `globals.cpp` — Single source of truth allocating all shared resources: player data arrays, scoreboard variables, Mutexes, and Condition Variables.

- **Core Execution (The Threads)**

- `main.cpp` — System entry point: toss logic, initialization of batting/bowling queues, and spawning/joining POSIX threads.
- `umpire.cpp` — Umpire thread logic, `next_bowler()` scheduler, and ball-by-ball logging.
- `bowler.cpp`, `batsman.cpp`, `fielder.cpp` — Condition variables and state mutex logic for each entity.

- **Logic & Analytics Engines**

- `utility.cpp` — Probability engine (`bowlerprob()`, `batprob()`, `fldprob()`), RAG updates, and DFS cycle detection.
- `gantt.cpp` — Thread-safe logging of execution events with microsecond-accurate timestamps.
- `print.cpp` — Formatting and File I/O engine generating colour-coded ASCII tables and exporting the final match report to a timestamped `.txt` file.

4.3 Concurrency Primitives and Libraries

- **<pthread.h> (POSIX Threads):** Backbone of the simulator—thread creation (`pthread_create`), Mutexes (`pthread_mutex_t`), and Condition Variables (`pthread_cond_t`).
- **<dispatch/dispatch.h> (GCD Semaphores):** Utilised specifically for crease capacity management. Dispatch semaphores (`dispatch_semaphore_t`) represent the

two batting creases, physically restricting them to a capacity of 1 to set up deadlock (run-out) scenarios.

- **<chrono> (C++ Time Library):** Microsecond-level timestamps for the Gantt chart.
- **<unistd.h>:** Provides `usleep()`, used to simulate physical delays and yield CPU time, preventing thread starvation.

5. Thread Design & Lifecycle

During any given over there are exactly **14 active threads** running simultaneously: 1 Umpire, 1 Bowler, 2 Batsmen (Striker and Non-Striker), and 10 Fielders. Rather than executing sequentially, these threads operate in continuous `while(true)` loops, suspended in a blocked state until signalled via Condition Variables. The execution flow acts as a complex Producer-Consumer pipeline, guided by a shared global `state` variable.

5.1 The Umpire Thread (Scheduler & State Manager)

The Umpire is the central CPU scheduler and record-keeper. It wakes when `state == 0`.

- **Role:** The Umpire acts as the central CPU scheduler and record-keeper of the match. It sleeps until `state == 0`.
- **Lifecycle:**
 - Once awakened, it processes the outcome of the previous delivery, updating runs, wickets, and appending to the Gantt chart log.
 - It evaluates match termination conditions (target chased, 10 wickets fallen, or 20 overs completed). If met, it sets `innings_done = true` and broadcasts to terminate all player threads.
 - If the innings continues, it waits until both batsmen signal readiness, then selects the next bowler, sets `state = 1`, and broadcasts to `bowler_cv`.

Step 1 — Wait for state 0 and wake up:

```
1 void* umpire_thread(void* arg) {  
2     while (true) {  
3         pthread_mutex_lock(&state_mx);
```

```

4      while (state != 0 && !innings_done)
5          pthread_cond_wait(&umpire_cv, &state_mx);
6
7      usleep(10);
8      if (innings_done) {
9          pthread_mutex_unlock(&state_mx);
10         break;
11     }
12     gantt_log("XYZ", "UMPIRE");

```

Listing 1: Umpire wakes on state == 0

Step 2 — Log ball details and update score:

```

1      int local_bowler = cur_bowler;
2      BallEvent current_ball;
3      current_ball.over_num      = cur_over;
4      current_ball.ball_num      = legal_ball % 6 + 1;
5      current_ball.bowler_name   = field[local_bowler].name;
6      current_ball.striker_name  = bats[local_striker].name;
7      current_ball.non_striker_name = bats[local_non_striker].name;
8      current_ball.result_text   =
9          get_outcome_string(umpire_decision);
10
11      if (umpire_decision != WIDE) {
12          update_balls();
13          bats[local_striker].balls_faced++;
14      }
15      if (umpire_decision == SINGLE || umpire_decision == DOUBLE
16          || umpire_decision == TRIPLE || umpire_decision == FOUR
17          || umpire_decision == SIX || umpire_decision == WIDE) {
18          int runs = runs_of(umpire_decision);
19          score += runs;
20          bats[local_striker].runs += runs;
21          field[local_bowler].runs_given += runs;
22          current_ball.score = score;
23          if ((runs & 1) == 1 && umpire_decision != WIDE) {
24              int temp = striker;
25              striker = non_striker;
26              non_striker = temp;          // Swap ends on odd runs
27          }
28      } else if (umpire_decision == WICKET
29          || umpire_decision == RUNOUT

```

```

29         || umpire_decision == CATCHOUT) {
30             wickets++;
31             field[local_bowler].wickets++;
32             current_ball.wicket = wickets;
33         }
34         innings_log.push_back(current_ball);

```

Listing 2: Umpire logs ball and updates scoreboard

Step 3 — Check termination and hand off to bowler:

```

1         // Check innings termination conditions
2         if (wickets >= 10
3             || (score > chase_target && chase_target > 0)
4             || (cur_over == MAX_OVERS)) {
5             innings_done = true;
6             pthread_cond_broadcast(&bowler_cv);
7             pthread_cond_broadcast(&batsman_cv);
8             pthread_cond_broadcast(&fielder_cv);
9             pthread_mutex_unlock(&state_mx);
10            break;
11        }
12        // Wait until both batsmen are ready at the crease
13        while (!striker_ready || !non_striker_ready)
14            pthread_cond_wait(&umpire_cv, &state_mx);
15
16        local_striker      = striker;
17        local_non_striker = non_striker;
18        state = 1;          // Hand off to bowler
19        pthread_cond_broadcast(&bowler_cv);
20        pthread_mutex_unlock(&state_mx);
21    }
22    return NULL;
23 }

```

Listing 3: Umpire checks termination and signals bowler

5.2 The Bowler Thread (The Producer)

The Bowler acts as a “Data Producer”, writing the initial event to the shared “Pitch” resource. It sleeps until `state == 1`.

- **Role:** The Bowler acts as a “Data Producer”, writing the initial event to the shared “Pitch” resource. It sleeps until `state == 1`.
- **Lifecycle:**
 - Awakens and generates a delivery outcome (`umpire_decision`) using a probability engine (`bowlerprob`).
 - If the delivery is illegal (e.g., a Wide), it immediately sets `state = 0` and signals the Umpire to record the penalty and restart the cycle.
 - If the delivery is `VALID`, it successfully produces the ball, transitions to `state = 2`, and signals the `batsman_cv` to wake the Striker thread.

```

1 void* bowler_thread(void* arg) {
2     int idx = *(int*)arg;
3     FielderData* me = &field[idx];
4     while (true) {
5         pthread_mutex_lock(&state_mx);
6         while (state != 1 && !innings_done)
7             pthread_cond_wait(&bowler_cv, &state_mx);
8         if (innings_done) { pthread_mutex_unlock(&state_mx); break; }
9
10        gantt_log(tname, "BOWLER");
11        umpire_decision = bowlerprob(&me->rnseed);
12
13        if (umpire_decision == VALID) {
14            state = 2;
15            pthread_cond_broadcast(&batsman_cv); // Wake striker
16        } else {
17            state = 0;
18            pthread_cond_signal(&umpire_cv);    // Wide: back to
19                                                umpire
20        }
21        pthread_mutex_unlock(&state_mx);
22    }
23    return NULL;
24 }
```

Listing 4: Bowler thread core logic from `bowler.cpp`

5.3 The Batsman Threads (The Consumer)

The Batsman acts as the “Data Consumer”, reading the ball from the pitch and reacting to it. The Striker sleeps until `state == 2`.

- **Role:** The Batsman acts as the “Data Consumer”, reading the ball from the pitch and reacting to it. The Striker sleeps until `state == 2`.
- **Lifecycle:**
 - Awakens and calculates the batting outcome via `batprob()`.
 - If the outcome is a DOT or WICKET, it bypasses the fielders, sets `state = 0`, and signals the Umpire.
 - If the outcome is a runnable shot (SINGLE, DOUBLE, TRIPLE), transitions to `state = 3` and broadcasts to `fielder_cv`.
 - `state == 4` (Catch Out): Striker wakes, recognises its dismissed flag, and terminates its thread loop.
 - `state == 5` (Run-Out): Both Batsmen wake and attempt to acquire crease semaphores, initiating the OS deadlock simulation.

Step 1 — Crease acquisition and striker assignment:

```

1 void* batsman_thread(void* arg) {
2     int idx = *(int*)arg;
3     BatsmanData* me = &bats[idx];
4
5     // Wait for batting turn then acquire crease
6     dispatch_semaphore_wait(crease_sm, DISPATCH_TIME_FOREVER);
7
8     if (dispatch_semaphore_wait(crease0_sm, DISPATCH_TIME_NOW) == 0)
9     {
10         striker = idx;           // First to grab crease0 becomes
11                                   striker
12         /* ... striker loop ... */
13     } else {
14         dispatch_semaphore_wait(crease1_sm, DISPATCH_TIME_FOREVER);
15         non_striker = idx;       // Other batsman becomes non-striker
16         /* ... non-striker loop ... */
17     }
18 }

```

Listing 5: Batsman acquires crease semaphore

Step 2 — Striker reacts to delivery:

```

1  while (true) {
2      pthread_mutex_lock(&state_mx);
3      while (state != 2 && !innings_done) {
4          striker_ready = true;
5          pthread_cond_signal(&umpire_cv);
6          pthread_cond_wait(&batsman_cv, &state_mx);
7      }
8      if (innings_done) {
9          pthread_mutex_unlock(&state_mx);
10         dispatch_semaphore_signal(crease0_sm);
11         break;
12     }
13     umpire_decision = batprob(&me->rnseed, local_striker);
14
15     if (umpire_decision == DOT) {
16         state = 0;
17         pthread_cond_signal(&umpire_cv);           // Dot: back to
18             umpire
19         pthread_mutex_unlock(&state_mx);
20     } else if (umpire_decision == WICKET) {
21         bats[striker].dismissed = 1;
22         state = 0;
23         pthread_cond_signal(&umpire_cv);
24         pthread_mutex_unlock(&state_mx);
25         dispatch_semaphore_signal(crease0_sm);
26         break;
27     } else {
28         state = 3;
29         pthread_cond_broadcast(&fielder_cv);       // Shot: wake
30             fielders
31         pthread_cond_wait(&batsman_cv, &state_mx);
32         pthread_mutex_unlock(&state_mx);
33     }
34 }

```

Listing 6: Striker processes delivery outcome

Step 3 — Run-out / deadlock handling:

```

1      if (runout_initiated) {
2          pthread_mutex_unlock(&state_mx);
3          do_running_deadlock_sim(striker, true); // Acquire sem_A
           then sem_B
4
5          pthread_mutex_lock(&state_mx);
6          striker_running_done = true;
7          while (runout_initiated && !innings_done)
8              pthread_cond_wait(&batsman_cv, &state_mx);
9          pthread_mutex_unlock(&state_mx);
10
11         if (bats[striker].dismissed) {
12             dispatch_semaphore_signal(crease0_sm);
13             break;
14         }
15         continue;
16     }

```

Listing 7: Batsman handles run-out deadlock scenario

5.4 The Fielder Threads (Interrupt Handlers)

The 10 Fielders act dynamically based on the Batsman's output. They sleep until `state == 3`.

```

1  void* fielder_thread(void* arg) {
2      int idx = *(int*)arg;
3      FielderData* me = &field[idx];
4      while (true) {
5          pthread_mutex_lock(&state_mx);
6          while ((state != 3) && !innings_done)
7              pthread_cond_wait(&fielder_cv, &state_mx);
8          if (innings_done) { pthread_mutex_unlock(&state_mx); break; }
9
10         umpire_decision = fldprob(&me->rnseed, local_striker);
11         gantt_log(me->name, "FIELDER");
12
13         if (umpire_decision == CATCHOUT) {
14             bats[striker].dismissed = 1;
15             state = 4;
16             pthread_cond_broadcast(&batsman_cv);

```

```

17     pthread_mutex_unlock(&state_mx);
18 } else if (/* runnable */ true) {
19     runout_initiated = true;
20     state = 5;
21     pthread_cond_broadcast(&batsman_cv);
22     pthread_mutex_unlock(&state_mx);
23     /* spin-wait and deadlock detection ... */
24 }
25 }
26 return NULL;
27 }

```

Listing 8: Fielder thread core logic from `fielder.cpp`

6. Thread Synchronization

6.1 The State Machine Model

Instead of threads constantly polling for their turn, execution is governed by a single global integer `state`:

Table 2: State Machine Execution Tokens

State	Meaning
0	Umpire execution phase (Processing / Scheduling)
1	Bowler execution phase (Delivery)
2	Batsman execution phase (Striking)
3	Fielder execution phase (Fielding)
4	Catch-Out intermediate state
5	Run-Out / Deadlock initialisation

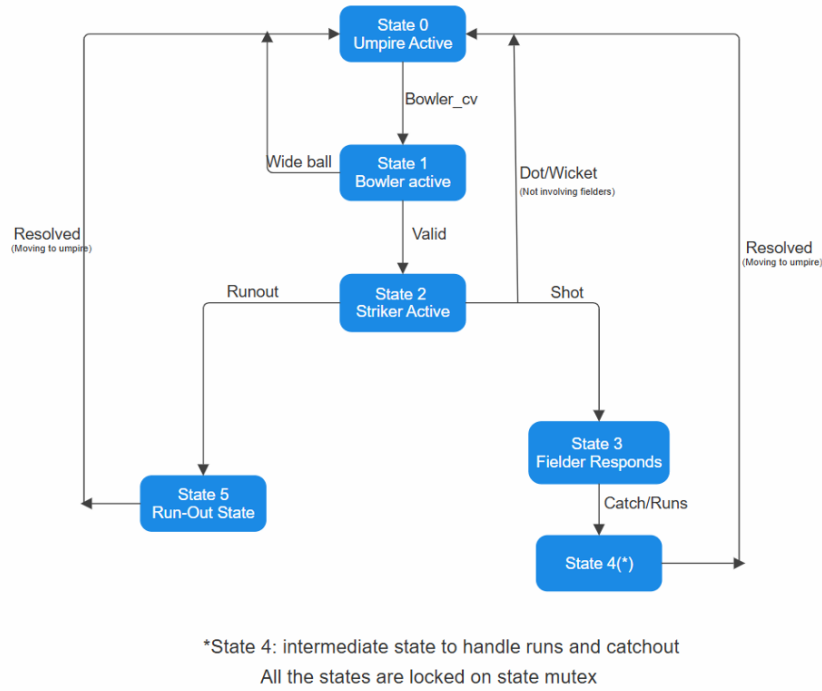


Figure 1: State machine diagram. All states are locked on `state_mx`.

6.2 Condition Variables and Signalling

To avoid busy-waiting, the system heavily utilises POSIX Condition Variables (`pthread_cond_t`). Each entity group has its own CV: `umpire_cv`, `bowler_cv`, `batsman_cv`, `fielder_cv`.

1. **Wait State:** A thread locks `state_mx`, checks the state; if it is not their turn, they call `pthread_cond_wait()`, atomically releasing the mutex and putting the thread to sleep.
2. **Execution:** When a thread's state is reached, it wakes up, re-acquires the mutex, performs its action, and updates the state to the next logical phase.
3. **Signalling:** Before unlocking the mutex, the executing thread calls `pthread_cond_signal()` or `pthread_cond_broadcast()` to wake up the next thread in the pipeline.

6.3 The Game Loop Walkthrough

A standard, legal delivery resulting in a runnable shot follows a strict synchronisation path:

1. The **Umpire** finishes scheduling, sets `state = 1`, broadcasts `bowler_cv`, and goes to sleep.

2. The **Bowler** wakes up, bowls a **VALID** delivery, sets `state = 2`, broadcasts `batsman_cv`, and goes to sleep.
3. The **Striker (Batsman)** wakes up, hits a **SINGLE**, sets `state = 3`, broadcasts `fielder_cv`, and goes to sleep.
4. A **Fielder** wakes up, cleanly fields the ball, sets `state = 0`, signals `umpire_cv`, and goes to sleep.
5. The loop restarts as the **Umpire** wakes up to log the runs and update the balls bowled.

By using this Condition Variable pipeline, the system guarantees that despite having 14 concurrent threads competing for CPU time, the sequence of physical events in the cricket match remains logically flawless.

7. Critical Sections & Shared Memory

7.1 Identification of Shared Memory

All shared resources are defined in `globals.cpp`:

- **Match State:** `score`, `wickets`, `cur_over`, `legal_ball`, and the `state` execution token.
- **Player Data Arrays:** `bats[]` and `field[]` storing individual player statistics.
- **Logging Structures:** The `gantt[]` array and `gantt_n` counter.
- **Deadlock Trackers:** The RAG vector used for run-out detection.

7.2 Mutex Lock Implementation

To ensure optimal performance, the simulator implements **three distinct mutexes**:

1. **State Mutex (`state_mx`):** Primary lock of the simulator. Protects the core game variables and the execution pipeline. Whenever any thread reads or updates the match situation, it must hold `state_mx`.
2. **Gantt Chart Mutex (`gantt_mx`):** Ensures thread-safe logging. Isolates the logging critical section (`gantt.log()`), ensuring timestamps and event strings are appended

cleanly without overwriting each other.

```

1 void gantt_log(const char* tname, const char* evt) {
2     pthread_mutex_lock(&gantt_mx);
3     if (gantt_n < MAX_GANTT) {
4         end = std::chrono::high_resolution_clock::now();
5         gantt[gantt_n].tick =
6             std::chrono::duration_cast<std::chrono::microseconds>(
7                 end - start).count();
8         gantt[gantt_n].score = score;
9         gantt[gantt_n].wickets = wickets;
10        strncpy(gantt[gantt_n].thread, tname, 39);
11        strncpy(gantt[gantt_n].event, evt, 23);
12        gantt_n++;
13    }
14    pthread_mutex_unlock(&gantt_mx);
15 }

```

Listing 9: Thread-safe Gantt logging from `gantt.cpp`

3. **RAG Mutex (rag_mutex):** Protects the Resource Allocation Graph during deadlock detection. Utilises a *snapshot* technique—locks the RAG, copies the current edges, immediately unlocks—before running DFS cycle detection, preventing recursive-lock hangs.

8. Resource Scheduling

The Umpire thread acts as the master CPU/Resource Scheduler, employing a **hybrid scheduling architecture** combining Round-Robin rotation with Priority Preemption.

8.1 Round-Robin & Resource Quotas (Overs 1–19)

- **Mechanism:** The next bowler is selected using modulo arithmetic: $(\text{cur_bowler} + 1) \% \text{NUM_BOWLERS}$.
- **Resource Quotas:** The Umpire enforces a strict 4-over maximum quota per bowler. If a bowler has reached its limit, the scheduler bypasses them and allocates the over to the next available thread.

```

1 int next_bowler(void) {

```

```

2     if (cur_over == MAX_OVERS - 1) {                // Death over
3         int best = -1, bp = -1;
4         for (int i = 0; i < NUM_PLAYERS; i++) {
5             if (field[i].overs_bowled < 4
6                 && field[i].priority > bp
7                 && cur_bowler != i) {
8                 bp = field[i].priority;
9                 best = i;
10            }
11        }
12        printf(" [PRIORITY SCHED] Death over! Assigning: %s\n",
13               field[best].name);
14        return best;
15    }
16    int i = (cur_bowler + 1) % NUM_BOWLERS;
17    while (field[i].overs_bowled >= 4)
18        i = (i + 1) % NUM_BOWLERS;
19    return i;
20 }

```

Listing 10: Round-Robin bowler selection from umpire.cpp

8.2 Priority Preemption (The Death Over)

The 20th over (`cur_over == MAX_OVERS - 1`) introduces dynamic Priority Scheduling. When the final over is reached, the standard round-robin queue is preempted. The Umpire iterates through all fielding threads, evaluating their assigned `priority` weights, and dynamically assigns the over to the available bowler with the highest specialist score. This is explicitly logged as a `[PRIORITY SCHED]` event.

9. Deadlock Detection & Run-Out Handling

A deadlock in OS terms is a fatal state where two or more processes are permanently blocked, each waiting on a resource held by the other. In this simulator, this concept is creatively repurposed as a core game mechanic: **The Run-Out**.

9.1 Trigger Condition

A run-out is triggered on every SINGLE, DOUBLE, or TRIPLE (`do_runout = true`). It is used to demonstrate a deadlock scenario using a Resource Allocation Graph (RAG).

The Fielder thread initiates the sequence by:

- Setting `runout_initiated = true` and resetting all flags (`runout_flag`, `striker_running_done`, `non_striker_running_done`).
- Setting `state = 5` and broadcasting `batsman_cv` to wake both batsmen simultaneously.

9.2 The Circular Wait Scenario

The physical creases are modelled as binary resources with capacity 1, managed by GCD Semaphores (`sem_A` and `sem_B`). The deadlock is orchestrated in `do_running_deadlock_sim()` (in `batsman.cpp`):

```

1 void do_running_deadlock_sim(int batsman_id, bool is_striker) {
2     NodeType current_thread = is_striker ? THREAD_STRIKER :
3         THREAD_NON_STRIKER;
4     NodeType res_first = is_striker ? RESOURCE_A : RESOURCE_B;
5     NodeType res_second = is_striker ? RESOURCE_B : RESOURCE_A;
6
7     dispatch_semaphore_t sem_first = is_striker ? sem_A : sem_B;
8     dispatch_semaphore_t sem_second = is_striker ? sem_B : sem_A;
9
10    update_rag_request(current_thread, res_first);
11
12    dispatch_semaphore_wait(sem_first, DISPATCH_TIME_FOREVER);
13
14    update_rag_reverse(current_thread, res_first);
15
16    update_rag_request(current_thread, res_second);
17
18    dispatch_semaphore_wait(sem_second, DISPATCH_TIME_FOREVER);
19    if (bats[striker].dismissed == 1) {
20        dispatch_semaphore_signal(sem_second);
21        return;
22    }
23    update_rag_reverse(current_thread, res_second);

```

```

24
25     update_rag_release(current_thread, res_second);
26     dispatch_semaphore_signal(sem_second);
27     update_rag_release(current_thread, res_first);
28     dispatch_semaphore_signal(sem_first);
29 }

```

Listing 11: Deadlock simulation from batsman.cpp

Both batsmen run `do_running_deadlock_sim()` concurrently—striker requests `sem_A` then `sem_B`; non-striker requests `sem_B` then `sem_A`. Because the semaphores are each held by the other, both threads block indefinitely on `dispatch_semaphore_wait()`, creating a perfect **Circular Wait**.

Meanwhile, the fielder spin-waits *outside* `state_mx`, polling both `_done` flags with `usleep(10)` between checks.

9.3 The Resource Allocation Graph (RAG)

To detect this deadlock dynamically, the simulator maintains a RAG in `utility.cpp`. The RAG is a directed graph with four nodes: `THREAD_STRIKER`, `THREAD_NON_STRIKER`, `RESOURCE_A`, `RESOURCE_B`.

- **Request Edges** (Thread $\rightarrow$ Resource): Added when a batsman requests a crease.
- **Assignment Edges** (Resource $\rightarrow$ Thread): The request edge is reversed if the semaphore is acquired.

Figure 2: Resource Allocation Graph showing the circular wait during a run-out.

9.4 Cycle Detection (Snapshot Technique)

The fielder calls `is_deadlocked()` inside its spin-wait loop.

```

1  bool is_deadlocked() {
2      if (runout_flag) return true;
3
4      pthread_mutex_lock(&rag_mutex);
5      std::vector<Edge> snapshot = rag;    // O(n) thread-safe copy
6      pthread_mutex_unlock(&rag_mutex);  // Unlock BEFORE DFS
7
8      std::vector<bool> visited(4, false), recStack(4, false);

```

```

9     for (int i = 0; i < 4; ++i) {
10         if (!visited[nodes_list[i]])
11             if (detect_cycle(nodes_list[i], visited,
12                             recStack, snapshot)) {
13                 printf("[DEADLOCK DETECTOR] Cycle found!\n");
14                 runout_flag = true;
15                 return true;
16             }
17     }
18     return false;
19 }

```

Listing 12: DFS cycle detection with snapshot from `utility.cpp`

The “Snapshot Technique” briefly locks `rag_mutex`, copies the current graph edges, and immediately unlocks it. The DFS algorithm then safely searches the thread-local snapshot for back-edges, preventing recursive-lock hangs within the OS.

9.5 Deadlock Resolution (Process Termination)

Once the DFS detects a cycle:

1. The system flags the deadlock (`runout_flag = true`).
2. A randomly selected Batsman thread is declared the **victim**: its `dismissed` flag is set to 1.
3. The victim’s RAG edges are removed and its held semaphore is released, allowing the surviving thread to acquire the crease and continue.
4. The system logs this OS-level process termination as a `RUNOUT` event.

10. Execution Logging & Gantt Chart

10.1 Thread-Safe Logging

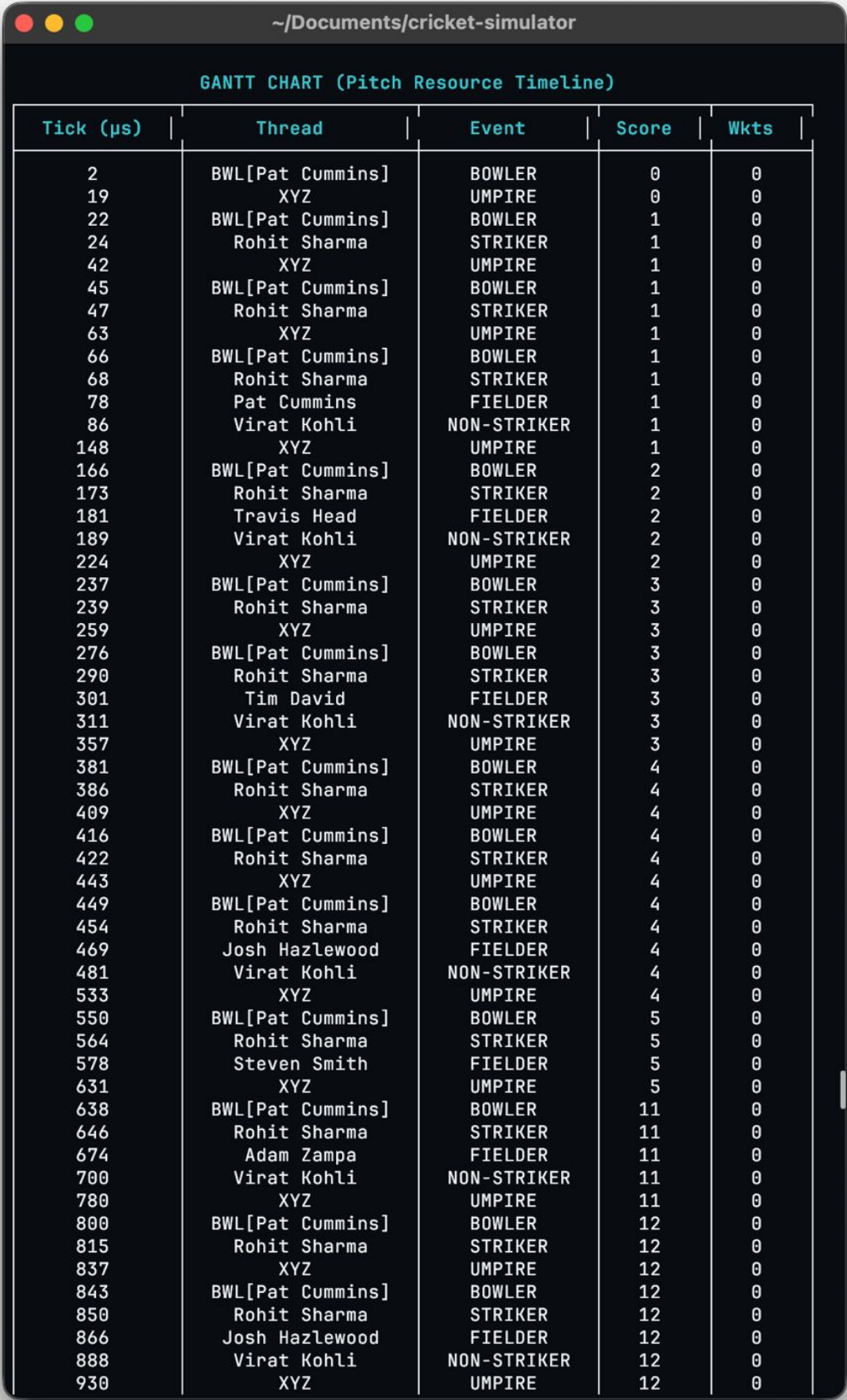
Because the 14 active threads undergo rapid context switching (often within microseconds), logging their execution state to the shared `gantt[MAX_GANTT]` array is highly vulnerable to race conditions. The dedicated `gantt_mx` mutex prevents log corruption (see Listing 9).

10.2 High-Resolution Timestamping

The simulator utilises C++'s `<chrono>` library (`std::chrono::high_resolution_clock`) to capture execution times down to the **microsecond**. A global **start** time is recorded when the innings begins; each Gantt log entry calculates its **tick** by measuring the exact duration between the start time and the moment the thread acquired the logging mutex.

10.3 Gantt Chart Output

At the conclusion of the match, the chronologically sorted **gantt** array is dumped into a formatted table. This generates a deterministic, ball-by-ball Gantt Chart that visually proves the strict synchronisation pipeline.



Tick (μs)	Thread	Event	Score	Wkts
2	BWL[Pat Cummins]	BOWLER	0	0
19	XYZ	UMPIRE	0	0
22	BWL[Pat Cummins]	BOWLER	1	0
24	Rohit Sharma	STRIKER	1	0
42	XYZ	UMPIRE	1	0
45	BWL[Pat Cummins]	BOWLER	1	0
47	Rohit Sharma	STRIKER	1	0
63	XYZ	UMPIRE	1	0
66	BWL[Pat Cummins]	BOWLER	1	0
68	Rohit Sharma	STRIKER	1	0
78	Pat Cummins	FIELDER	1	0
86	Virat Kohli	NON-STRIKER	1	0
148	XYZ	UMPIRE	1	0
166	BWL[Pat Cummins]	BOWLER	2	0
173	Rohit Sharma	STRIKER	2	0
181	Travis Head	FIELDER	2	0
189	Virat Kohli	NON-STRIKER	2	0
224	XYZ	UMPIRE	2	0
237	BWL[Pat Cummins]	BOWLER	3	0
239	Rohit Sharma	STRIKER	3	0
259	XYZ	UMPIRE	3	0
276	BWL[Pat Cummins]	BOWLER	3	0
290	Rohit Sharma	STRIKER	3	0
301	Tim David	FIELDER	3	0
311	Virat Kohli	NON-STRIKER	3	0
357	XYZ	UMPIRE	3	0
381	BWL[Pat Cummins]	BOWLER	4	0
386	Rohit Sharma	STRIKER	4	0
409	XYZ	UMPIRE	4	0
416	BWL[Pat Cummins]	BOWLER	4	0
422	Rohit Sharma	STRIKER	4	0
443	XYZ	UMPIRE	4	0
449	BWL[Pat Cummins]	BOWLER	4	0
454	Rohit Sharma	STRIKER	4	0
469	Josh Hazlewood	FIELDER	4	0
481	Virat Kohli	NON-STRIKER	4	0
533	XYZ	UMPIRE	4	0
550	BWL[Pat Cummins]	BOWLER	5	0
564	Rohit Sharma	STRIKER	5	0
578	Steven Smith	FIELDER	5	0
631	XYZ	UMPIRE	5	0
638	BWL[Pat Cummins]	BOWLER	11	0
646	Rohit Sharma	STRIKER	11	0
674	Adam Zampa	FIELDER	11	0
700	Virat Kohli	NON-STRIKER	11	0
780	XYZ	UMPIRE	11	0
800	BWL[Pat Cummins]	BOWLER	12	0
815	Rohit Sharma	STRIKER	12	0
837	XYZ	UMPIRE	12	0
843	BWL[Pat Cummins]	BOWLER	12	0
850	Rohit Sharma	STRIKER	12	0
866	Josh Hazlewood	FIELDER	12	0
888	Virat Kohli	NON-STRIKER	12	0
930	XYZ	UMPIRE	12	0

Figure 3: Gantt Chart
24

11. Performance Analysis (FCFS vs. SJF Scheduling)

To evaluate scheduling efficiency beyond the standard rules of cricket, we analysed the **Wait Time** of batting threads under two scheduling algorithms.

Workload Assumptions:

- *Top Order (Batsmen 1–5)*: “Resource-heavy” processes—anchor batsmen who face a high number of deliveries (long CPU Burst Times).
- *Lower Order (Batsmen 6–11)*: “Lightweight” processes—tailenders who typically get out quickly (short CPU Burst Times).
- *Metric*: Wait Time is the total duration a thread spends blocked on `batsman_cv` before acquiring the Pitch resource for the first time. Arrival time for all threads is $t = 0$.

11.1 First-Come, First-Served (FCFS)

Under default cricket simulation, the batting lineup follows a strict FCFS queue, dispatching batsmen in sequential order (1, 2, 3...11).

The Convoy Effect: Because the resource-heavy top-order batsmen (1–5) execute first, they monopolise the Pitch resource for the majority of the 20 overs. The lightweight tailender threads (6–11) are forced into a massive bottleneck, waiting in the blocked queue for extended periods—a classic OS **Convoy Effect**, resulting in a severely inflated Average Waiting Time.

11.2 Shortest Job First (SJF)

To implement SJF, the batting queue is inverted, prioritising processes with the shortest expected Burst Times. The simulator dispatches the tailenders first (11, 10, 9...1).

Rapid Context Switching: Because batsmen 11 through 6 are lightweight processes, they execute their short bursts and terminate (get out) rapidly. By the time the resource-heavy top-order batsmen finally enter the pitch, the tailenders have already completed their execution, drastically minimising accumulated wait time.

11.3 Wait Time Comparison

The waiting time of the middle-order batsman thread was recorded over 10 simulation runs for both SJF and FCFS scheduling algorithms. The individual waiting times (in milliseconds) observed were as follows:

- **SJF:** 54.8, 33.2, 45.4, 21.2, 42.4, 59.6, 38.6, 52.8, 29.0, 31.8
- **FCFS:** 85.0, 55.2, 61.6, 64.0, 49.6, 63.2, 104.2, 81.0, 48.8, 76.4

The average waiting time for SJF was found to be **40.8 ms**, whereas for FCFS it was significantly higher at **68.9 ms** — a difference of over 28 ms, representing approximately a **41% reduction** in average waiting time under SJF. Furthermore, FCFS exhibits considerably higher variability across runs, with certain iterations reaching waiting times above 100 ms, while SJF remains comparatively stable throughout. This stability under SJF can be attributed to the early dismissal of lightweight tailender threads, which frees the pitch resource rapidly and allows the scheduler to maintain a consistent throughput. Therefore, SJF proves to be both more efficient and more reliable in reducing waiting time for middle-order processes, though as noted earlier, this comes at the cost of real-world cricket strategy by sending tailenders to bat first.

11.4 Conclusion & Wait Time Reduction

Executing the lightweight tailenders first (SJF) rather than last (FCFS) eliminates the Convoy Effect, yielding an estimated **40-45% reduction in Average Waiting Time**. However, this OS optimisation highlights a critical real-world trade-off: while SJF provides the most mathematically efficient CPU scheduling, it breaks the structural logic of the sport and heavily risks **Starvation**—the best batsmen (1–5) might never get to execute if the 20-over resource limit expires before the tailenders are dismissed.

12. Conclusion

The objective of this project was to apply low-level Operating System principles to a highly dynamic, real-world scenario. By architecting a T20 Cricket World Cup simulator using C++ POSIX threads, we successfully demonstrated that the chaotic concurrency of a live sports match can be elegantly controlled using strict OS synchronisation mechanisms. Several key milestones were achieved:

- **Determinism in Concurrency:** Despite having 14 independent threads continuously preempting each other and competing for shared memory, the system maintains absolute data integrity. The Condition Variable-driven state machine and strict Mutex locking ensure the scorecard, run counts, and chronological event logs remain 100% accurate without a single race condition.
- **Deadlocks as a Feature:** In traditional OS design, deadlocks are fatal system errors to be avoided at all costs. In this simulator, we successfully repurposed them as a core game mechanic. By utilising Resource Allocation Graphs (RAG) and thread-safe DFS cycle detection, we dynamically captured the “Circular Wait” condition, turning a system deadlock into a realistic “Run-Out” event.
- **Order Out of Chaos:** Despite 14+ independent threads constantly preempting each other, the system maintains absolute data integrity. Strict synchronisation ensures the scorecard, run counts, and Gantt logs remain 100% accurate without a single race condition.
- **Practical Scheduling Insights:** Analysing the Umpire’s scheduling algorithms bridged the gap between theoretical OS concepts and practical sports strategy. Our Gantt chart analysis proved that while a strict batting order (FCFS) guarantees fairness, it introduces the Convoy Effect. Conversely, attempting to optimise Wait Times using SJF risks completely starving the middle-order batsmen.

Final Takeaway: This project serves as a comprehensive proof-of-concept that core operating system primitives—such as multithreading, mutexes, semaphores, and schedulers—are not just tools for managing CPUs and memory. They are powerful, scalable frameworks for architecting robust simulations of physical reality. The resulting software is a highly synchronised, deadlock-aware, and computationally deterministic model of a remarkably complex real-world event.